Distributed Execution Engine

Project Architecture & Technical Overview

Executive Summary

The Algorithmic Remote Code Execution (RCE) Engine is a production-grade, distributed microservices platform designed to safely compile and execute untrusted C++ code. Moving beyond traditional monolithic architectures, this system leverages a scalable task-queue topology to process asynchronous workloads. It completely isolates code execution using ephemeral Docker-in-Docker (DinD) sandboxes and delivers real-time telemetry back to the client using Server-Sent Events (SSE). The infrastructure is deployed on Kubernetes and features automated horizontal scaling to maintain low latency during periods of high traffic.

Technology Stack

Domain	Technology	Purpose in Architecture
Frontend	Vanilla JS, Monaco Editor	Provides a native IDE experience with syntax highlighting and code capture.
API Gateway	Python, Flask	Ingests code submissions, routes tasks, and maintains SSE streams.
Task Brokering	Redis, Celery	Decouples HTTP requests from heavy compilation workloads via a message queue.
Database	MySQL	Acts as the source of truth for job states, preventing race conditions.
Infrastructure	Kubernetes, Docker (DinD)	Orchestrates pods, load balances traffic, and enforces strict security sandboxing.

Key Engineering Achievements

- **Zero-Polling Delivery:** Replaced standard HTTP polling loops with a persistent unidirectional SSE stream, dramatically reducing network overhead.
- **Dynamic Auto-scaling:** Configured Kubernetes Horizontal Pod Autoscaler (HPA) to monitor live worker CPU usage and provision new nodes on demand.
- **Race-Condition Resiliency:** Implemented database reconciliation strategies to ensure UI consistency even when execution outpaces stream initialization.

System Architecture Topography



Request Lifecycle

1. **Code Ingestion:** The user submits code through the Monaco Editor. The payload is sent via HTTP POST to the Flask REST API.
2. **State Initialization:** The API logs a `Queued` state into the MySQL database to track the job, then delegates the heavy execution payload to the Redis message broker using Celery.
3. **Worker Activation:** A free Celery worker pulls the task from Redis. It immediately publishes a `Compiling Docker Container...` status message to a specific Redis Pub/Sub channel identified by the Job ID.
4. **Secure Sandboxing:** The worker utilizes Docker-in-Docker (DinD). It mounts the user's C++ code into a pristine, ephemeral container, executes the `g++` compilation process, passes in standard input, and captures the resulting standard output.
5. **Real-Time Streaming:** Concurrently, the Flask API opens a Server-Sent Events (SSE) generator function for the client. It subscribes to the worker's Redis Pub/Sub channel and yields status updates to the browser the millisecond they are published.
6. **Teardown & Persistence:** Upon execution completion, the worker destroys the ephemeral container and publishes the final output. The Flask API saves this final state to MySQL and safely terminates the SSE connection.

Core Engineering Solutions

Building a robust distributed system requires handling edge cases where microservices fall out of sync. Below are the primary architectural challenges encountered and the solutions implemented.

1. Real-Time Streaming & The Pub/Sub Race Condition

The Problem: Traditional HTTP polling (requesting status every 1 second) was inefficient. We upgraded to Server-Sent Events (SSE) via Redis Pub/Sub. However, if a C++ script compiled in 50ms, the worker published the "Finished" signal *before* the browser's SSE connection was fully established, causing the frontend to listen to an empty channel indefinitely.
The Solution: Implemented a Database Reconciliation strategy. Before the Flask API subscribes to the Redis stream, it queries MySQL. If the job is already marked terminal (SUCCESS/FAILURE), it bypasses the stream and instantly yields the final database result. If it is still processing, it falls back to the live Redis stream.

2. DinD Cold Boots & Pre-fetching Bottlenecks

The Problem: The initial Docker-in-Docker startup sequence takes considerable time to pull layers. During one test, a worker took over 100 seconds to cold-boot. Because Celery aggressively "pre-fetches" tasks, a frozen or slow worker would kidnap tasks from the Redis queue, leaving healthy workers idle while the user waited.
The Solution: Configured Celery worker concurrency rules to disable aggressive pre-fetching, ensuring tasks remain in the Redis broker until a worker is genuinely idle and fully booted, preventing ghost-queueing.

3. Cloud-Native Scalability via HPA

The Problem: An influx of complex `while(true)` loops from multiple clients would peg the single worker container to 100% CPU, causing subsequent code submissions to backlog in the Redis queue and timeout.
The Solution: Deployed the Kubernetes Metrics Server and defined strict CPU baselines (requests/limits) for the Celery pods. Implemented a Horizontal Pod Autoscaler (HPA) configured to automatically provision up to 3 replica worker nodes whenever average CPU utilization crosses the 50% threshold, ensuring high availability during traffic spikes.

Operations & Restart Guide

Use the following command sequence to reliably spin up the entire distributed environment from a powered-off state, or to bounce the system after making architectural code changes.

Action	CLI Command
1. Boot Virtual Machine	`minikube start`
2. Enable Telemetry (Required for HPA auto-scaling)	`minikube addons enable metrics-server`
3. Build Python Code (Must run inside project directory)	`docker build -t rce-backend:latest .`
4. Inject Image into K8s	`minikube image load rce-backend:latest`
5. Restart K8s Pods (Forces pods to pull the new image)	`kubectl rollout restart deployment flask-api celery-worker`
6. Expose API to Browser	`minikube service flask-api --url`

System Monitoring Commands (Troubleshooting)

If the system becomes unresponsive or streams are silent, use these commands to diagnose the cluster:

- **Check Pod Status:** `kubectl get pods` (Ensure all are "Running")
- **View Auto-scaler:** `kubectl get hpa -w` (Watch live CPU metrics)
- **Read API Logs:** `kubectl logs deployment/flask-api --tail=20`
- **Read Worker Logs:** `kubectl logs deployment/celery-worker -c celery --tail=50`